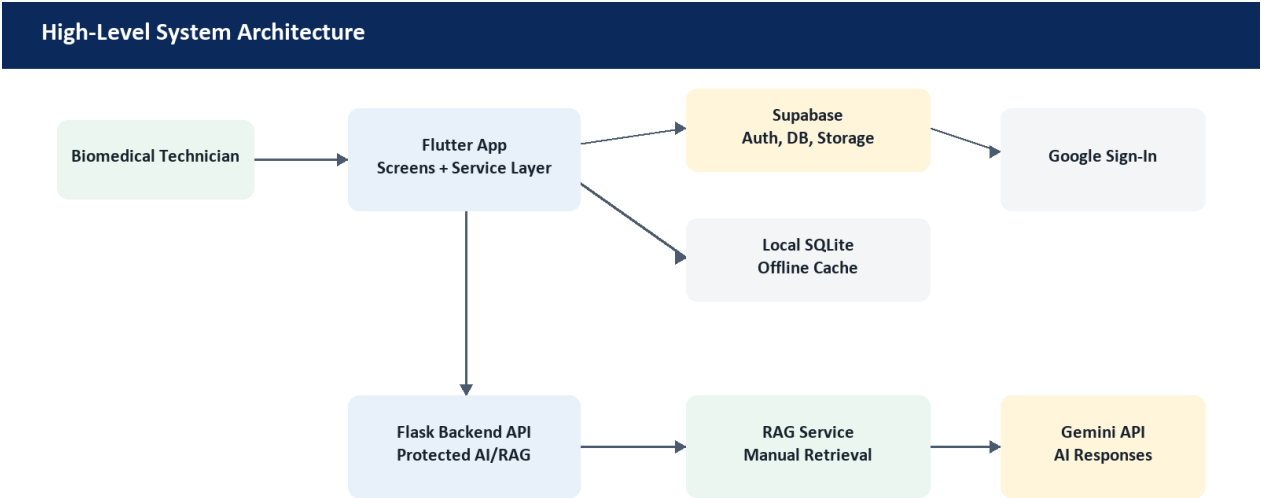


Pulse Clinical Equipment Maintenance Platform

Architecture and Crucial Implementation Documentation

Pulse is a Flutter-based clinical equipment maintenance platform for biomedical technicians. It combines mobile-first workflows, authenticated cloud services, offline storage, AI diagnostics, manual retrieval, and collaboration tools into one operational system.

Figure 1. System architecture at a glance



1. Architecture Overview

The project is organized into a layered architecture. Flutter owns the user experience, services own business logic and integration, Supabase owns identity and shared cloud data, SQLite provides offline resilience, and the Flask backend protects AI/RAG operations behind verified Supabase sessions.

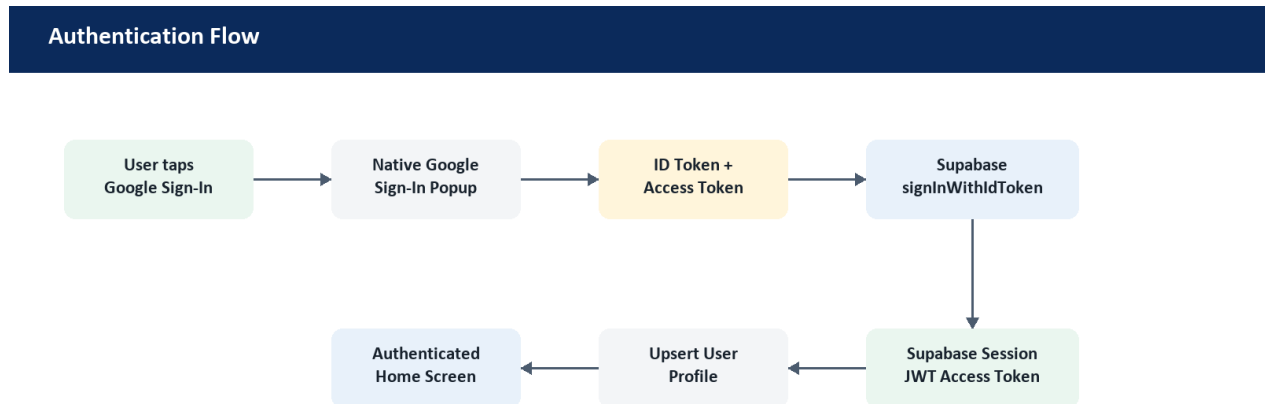
- Presentation layer: screens and widgets for dashboard, assets, manuals, AI assistant, collaboration, and settings.
- Service layer: authentication, database access, AI, RAG, synchronization, notifications, and collaboration.
- Persistence layer: local SQLite cache plus Supabase tables and storage.
- Backend layer: Flask endpoints that validate JWTs before executing AI or manual-retrieval workflows.

2. Frontend Application Flow

The app starts in `main.dart`, initializes Supabase, checks the cached session, and routes the user to either `LoginScreen` or `HomeScreen`. Once authenticated, `HomeScreen` becomes the hub for dashboard, asset analytics, manuals, AI assistant, collaboration, profile, and settings features.

3. Authentication and Session Handling

Figure 2. Google authentication flow



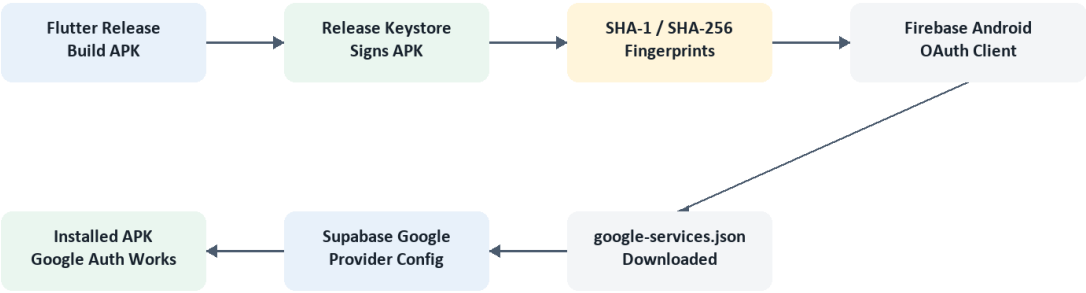
Authentication is centralized in AuthService. Email/password and Google Sign-In both resolve into Supabase sessions, which means the rest of the app and backend can trust one session model. For Google auth, the app obtains an ID token from Google and exchanges it through Supabase using `signInWithIdToken`.

- Supabase remains the source of truth for current user and session state.
- User profiles are synchronized into the users table after successful login or registration.
- Cached sessions are validated at startup so protected screens are not shown to invalid sessions.
- The backend accepts only Supabase JWT bearer tokens for protected AI/RAG operations.

4. Android Google Auth Release Reliability

Figure 3. Release signing and Google OAuth chain

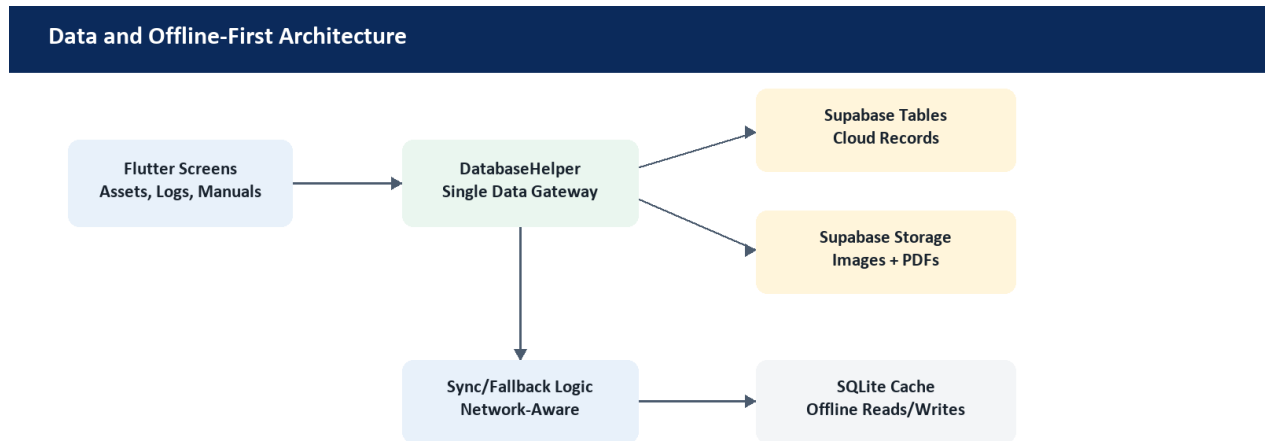
Android Google Sign-In Release Build Chain



Installed Android APKs require the package name, signing certificate fingerprint, Firebase Android OAuth client, google-services.json, and Supabase Google provider credentials to agree. This is the critical production step that prevents Google Sign-In from failing after installation on a real device.

5. Data and Offline-First Strategy

Figure 4. Data architecture



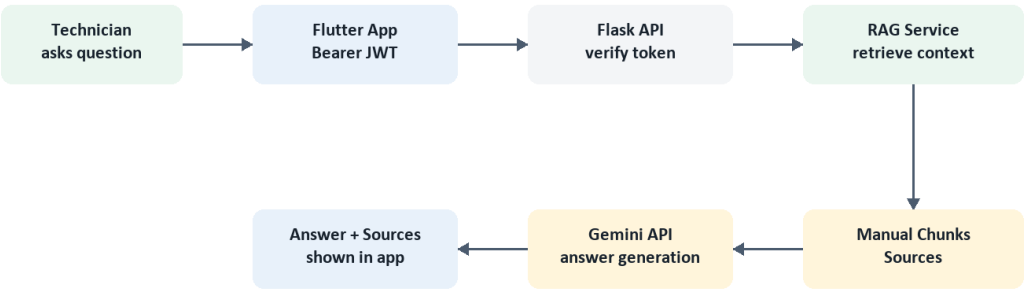
DatabaseHelper acts as the main gateway between UI workflows and persistence. The app can read and write from SQLite for local resilience while using Supabase tables and storage for authenticated cloud records, inventory images, manuals, and shared maintenance data.

- SQLite gives the technician fast local access and offline fallback.
- Supabase tables provide cloud-backed assets, spare parts, service logs, manuals, and user data.
- Supabase Storage stores binary assets such as inventory images and manual PDFs.
- Fallback logic keeps clinical workflows usable even when the network is unstable.

6. AI Assistant and RAG Workflow

Figure 5. AI/RAG request lifecycle

AI and RAG Request Flow



The AI assistant uses Gemini for diagnostic support and a backend RAG service for source-backed manual answers. The Flutter app sends the current Supabase access token to the Flask backend, where the token is verified before any retrieval, indexing, or AI generation work is allowed.

7. Crucial Modules

Layer	Main Files	Responsibility
App shell	main.dart	Initialize Supabase, validate session, route to login or home.
Auth	auth_service.dart	Email auth, Google auth, password reset, session cleanup, user profile sync.
Data	database_helper.dart	SQLite cache, Supabase CRUD, storage uploads, offline fallback.
AI/RAG	rag_api_service.dart, backend/app.py, backend/rag_service.py	Protected manual retrieval and Gemini integration.
Collaboration	chat_service.dart, google_chat_service.dart	Team communication, meetings, and Google Chat notifications.
Operations	notification_service.dart, sync_service.dart	Local notifications, background behavior, sync strategy.

8. Security Model

The security model is built around authenticated sessions and server-side verification. Client-side state is never enough for sensitive backend operations. Supabase row-level security and storage policies limit access to authenticated users and user-scoped paths, while Flask protects AI and RAG endpoints with JWT validation.

- Supabase Auth controls application sessions.
- Backend APIs verify bearer tokens before executing protected operations.
- Storage uploads are scoped to authenticated user folders.
- Secrets and signing keys are kept out of source control through local config files.

9. Why This Design Is Robust

The project follows an industry-style split of responsibilities: Flutter is focused on interaction, services handle integration and domain logic, Supabase centralizes identity and data, SQLite protects field usability, and Flask keeps expensive AI workflows behind server-side authorization. This makes the system maintainable, secure, and resilient for real clinical maintenance environments.